

Design and Physical Implementation of a Tiny Neural Network Accelerator with an 8×8 Systolic MAC Array

Govardhana Kondapaturi
Electrical and Computer Engineering
California State University, Fresno
California, USA
srigovardhan96@gmail.com

Abstract—This project describes the RTL-to-GDSII implementation of a systolic array based neural network accelerator for fast matrix multiplication. The design uses a grid of multiply-accumulate (MAC) processing elements (PEs), where each PE includes a pipelined MAC that separates multiplication and accumulation into different pipeline stages to reduce the critical path and support a higher clock frequency. An 8x8 systolic array interconnects these PEs to enable parallel data flow computation, and a small neural-network layer module adds bias and applies ReLU to demonstrate an inference-style data path. The RTL is functionally verified in ModelSim and synthesized using Synopsys Design Compiler to obtain timing, area, and power reports; pipelining is then applied inside the PEs and the design is re-synthesized to improve the critical path. Physical implementation is carried out in Cadence Innovus through design import, global-net connection, floorplanning, power-ring/strip creation, special routing, and connectivity checks, followed by standard-cell placement and pre-CTS optimization. A key step is power rail analysis performed after placement, where Innovus power reports are used along with synthesis power as a reference to evaluate the IR drop Analysis and adjust the power rails if required before proceeding to clock tree synthesis, routing, DRC verification and filler insertion for tape-out readiness.

Index Terms—Systolic Array, Matrix Multiplication, Neural Network Accelerator, Multiply-Accumulate (MAC), Processing Element (PE), Pipelining, Power rails, IR drop Analysis

I. INTRODUCTION

Matrix multiplication is the dominant computation in many neural network workloads and is fundamentally composed of repeated multiply-accumulate (MAC) operations [3]. A MAC performs the essential neural-network primitive in multiplying an input by a weight and adding the product into an accumulator and this operation is executed at extremely high volume across layers. As a result, neural network acceleration highly depends on how efficiently MAC computations can be performed in parallel.

A proven architecture for achieving high performance is the systolic array, in which many small compute blocks, known as processing elements (PEs), are arranged in a two-dimensional grid. Each PE performs local MAC operations while forwarding data to neighboring PEs in a synchronized and regular pattern. This streaming data flow structure enables

high throughput because, once the array is filled, a large number of MAC operations can be completed every clock cycle with minimal global communication. The effectiveness of this approach is demonstrated by modern AI accelerators such as Google’s Tensor Processing Unit (TPU), which leverages systolic data movement and massive MAC parallelism [2] to efficiently support neural network inference.

Beyond matrix multiplication, practical neural-network layers also require activation functions to introduce non-linearity. One of the most widely used activations is ReLU (Rectified Linear Unit) [4], which sets negative values to zero and passes positive values unchanged. Although ReLU is computationally simple, integrating it directly with the MAC outputs supports an inference-ready datapath by producing final activated results without additional external processing.

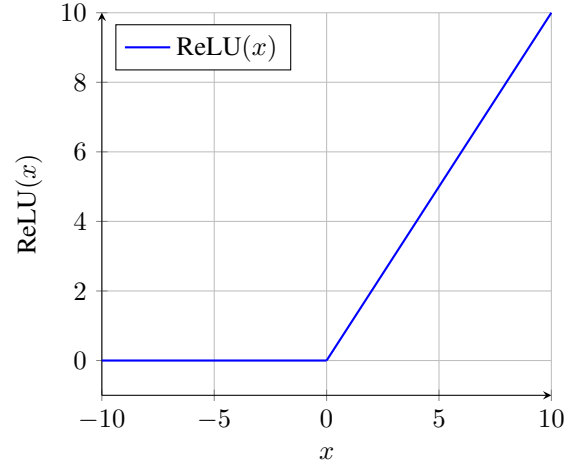


Fig. 1. ReLU Activation Function

In this work, an 8x8 systolic array is constructed from pipelined MAC-based PEs to demonstrate parallel matrix multiplication suitable for neural network acceleration. Bias addition and ReLU activation are included to represent complete neural-network layer output, emphasizing how PE-level MAC computation and systolic dataflow together enable a high-throughput inference-oriented hardware design.

II. LITERATURE SURVEY

Systolic array architectures have been widely discussed in VLSI and computer architecture literature as an efficient model for implementing arithmetic-intensive computations such as matrix multiplication. Kung and Leiserson introduced systolic arrays as regular, grid-based structures [1] in which processing elements communicate only with their neighbors, resulting in predictable dataflow and reduced global interconnect complexity. Textbooks on VLSI system design highlight that this regularity makes systolic arrays scalable and well suited for high-throughput hardware implementations.

Multiply–accumulate (MAC) units form the core computational block in many signal processing and neural network systems. Digital integrated circuit design literature emphasizes that the performance of MAC-dominated architectures is strongly influenced by critical path delay and operating frequency. Pipelining is identified as an effective technique to improve MAC performance by dividing long combinational paths into shorter stages, thereby enabling higher clock speeds while maintaining stable operation [6].

From a neural network perspective, activation functions are required to introduce non-linearity following linear MAC operations. Deep learning textbooks identify the Rectified Linear Unit (ReLU) as one of the most commonly used activation functions due to its simplicity and hardware friendliness [5]. Since ReLU requires only basic comparison logic, it can be efficiently integrated into hardware datapaths with minimal overhead. These established concepts from literature form the basis for designing efficient systolic-array-based neural network accelerators [7].

III. OBJECTIVES

This project primarily aims to realize a systolic-array–based neural network accelerator [7] through a complete RTL-to-GDSII design flow. To accomplish this goal, the following specific objectives are defined:

- 1) To design a pipelined multiply–accumulate (MAC) based processing element (PE) suitable for high-throughput computation.
- 2) To construct an 8×8 systolic array architecture enabling parallel, streaming dataflow for matrix multiplication.
- 3) To integrate bias addition and Rectified Linear Unit (ReLU) activation to demonstrate an inference-style datapath.
- 4) To analyze and improve timing, area, and power through synthesis and pipelining.
- 5) To evaluate physical design feasibility, including power delivery and layout feasibility considerations, DRC check prior to tape-out.

IV. ARCHITECTURE AND DESIGN REVIEW

The proposed accelerator architecture is composed of three hierarchical RTL modules: a pipelined processing element (PE_MAC), an 8×8 systolic array constructed from these processing elements, and a top-level neural network layer that integrates bias addition and activation functionality. The

architecture is derived directly from the RTL implementation and is designed to support matrix multiplication and inference-style computation.

A. Processing Element (PE_MAC)

The PE_MAC module implements a pipelined multiply–accumulate operation. Each processing element accepts two signed inputs, a_{in} and b_{in} , and maintains an internal accumulator register. The MAC operation is divided into two sequential pipeline stages. In the first stage, the input operands are multiplied and the product is stored in a pipeline register. In the second stage, the stored product from the previous cycle is added to the accumulator register.

Mathematically, the operation performed by each PE can be expressed as:

$$\text{prod_reg}(n+1) = a_{in}(n) \times b_{in}(n)$$

$$\text{acc}(n+1) = \text{acc}(n) + \text{prod_reg}(n)$$

The accumulator can be synchronously cleared using a control signal, allowing the same PE to be reused for multiple matrix operations. In addition to computation, each PE forwards its input operands to neighboring elements through registered outputs. The a operand propagates along one dimension of the array, while the b operand propagates along the orthogonal dimension, enabling localized data movement.

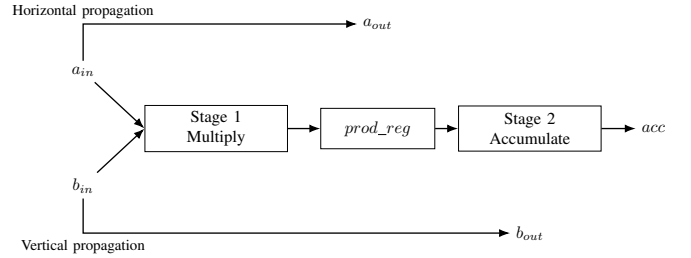


Fig. 2. PE_MAC block diagram showing two-stage pipelined MAC operation and operand forwarding.

B. Systolic Array Organization

The systolic array module instantiates 64 PE_MAC units arranged in an 8×8 grid. Input operands are provided to the array as packed buses and are unpacked and injected at the array boundaries. One set of operands is injected along the first row of the array, while the second set is injected along the first column.

Each processing element receives operands from its neighboring elements and performs one multiply–accumulate operation per clock cycle once the pipeline is filled. The accumulation performed within each PE corresponds to the matrix multiplication operation:

$$C_{i,j} = \sum_k A_{i,k} \times B_{k,j}$$

where the summation is realized temporally as operands propagate through the array over successive cycles. The accumulated results are stored locally within each PE and are collected into an output matrix.

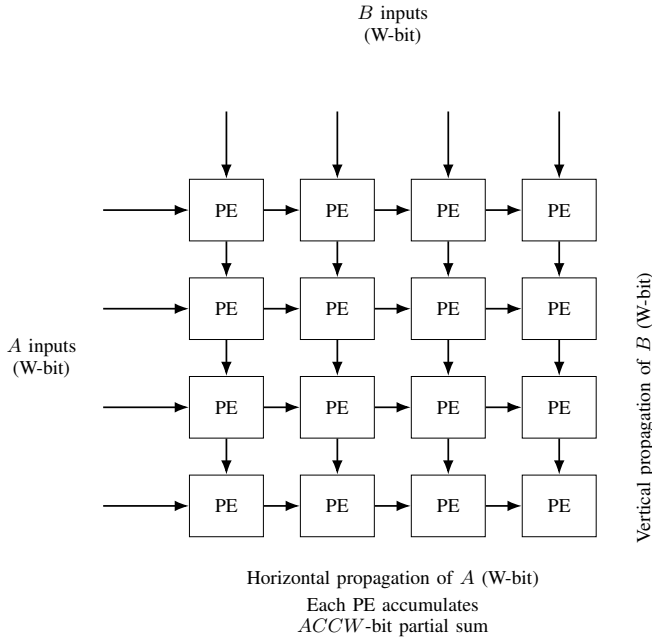


Fig. 3. Conceptual systolic array dataflow (shown as 4×4 for clarity; implemented as 8×8). Each processing element performs pipelined MAC operations with horizontal propagation of A operands and vertical propagation of B operands.

C. Top-Level Neural Network Layer

The top-level neural network layer integrates the systolic array with bias addition and activation logic. The raw output of the systolic array represents the matrix multiplication result. A bias value is added to each row of the output matrix using combinational logic.

Following bias addition, a Rectified Linear Unit (ReLU) activation function is applied to each output element. The ReLU operation outputs zero for negative values and passes positive values unchanged, and the final results are registered to ensure synchronous operation. This module produces activation-ready outputs corresponding to a single neural network layer.

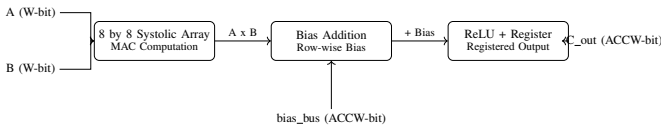


Fig. 4. Top-level tiny neural network layer datapath implementing matrix multiplication followed by bias addition and ReLU activation using an 8×8 systolic array.

V. IMPLEMENTATION

A. Verifying Verilog Code

Designing and implementing the proposed systolic-array-based neural network accelerator requires thorough functional verification of the RTL design before proceeding to synthesis. The RTL is written in Verilog using a modular and hierarchical approach, consisting of a pipelined processing element with 8-bit signed input operands and a 32-bit accumulator, an 8×8

systolic array constructed from these processing elements, and a top-level neural network layer that integrates row-wise bias addition and ReLU activation.

Functional verification is carried out using ModelSim with a comprehensive, self-checking testbench. During simulation, 8-bit input operands are streamed into the systolic array in a cycle-by-cycle manner consistent with the systolic dataflow architecture, while 32-bit bias values are applied to each output row. The processing elements accumulate partial products into 32-bit output elements, and the complete output matrix is packed onto a wide output bus containing all 64 elements. The observed waveforms confirm correct multiply-accumulate behavior, proper accumulation across clock cycles, and correct handling of pipeline latency and flush cycles. After bias addition, the ReLU activation correctly clamps negative values to zero, and the final activated outputs match the golden reference values computed in the testbench. These results demonstrate that the Verilog implementation is functionally correct and ready for synthesis and further implementation stages.

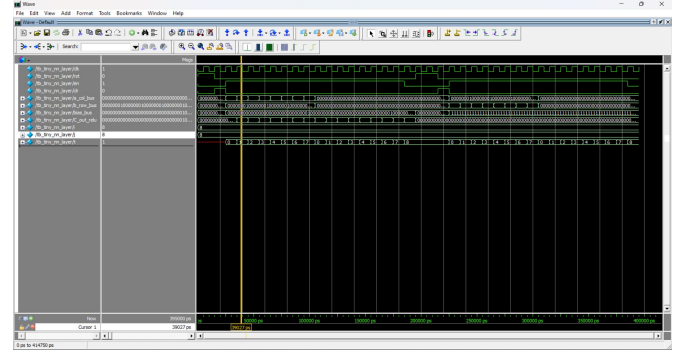


Fig. 5. ModelSim waveform confirm correct systolic data propagation, multiply-accumulate behavior across cycles, bias addition, and ReLU activation

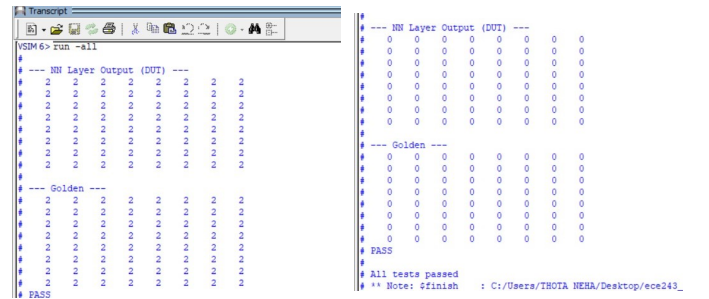


Fig. 6. ModelSim simulation transcript showing comparison between DUT output and Golden References with all test being passed.

The simulation transcript above shows that for multiple test cases, including positive and negative bias scenarios, the hardware output of the tiny neural network layer exactly matches the golden reference computed in the testbench. This confirms correct matrix multiplication, bias addition, ReLU activation, and proper handling of pipeline latency in the systolic array architecture.

B. Synthesis

After successful functional verification, the RTL design of the tiny neural network accelerator is synthesized using Synopsys Design Compiler with the Nangate Open Cell Library. During synthesis, the hierarchical Verilog description is translated into a gate-level netlist by mapping the design onto standard cells while satisfying the specified timing constraints. This process enables quantitative evaluation of the design in terms of area, power, and timing.

The area report indicates a total cell area of approximately $77,307 \mu\text{m}^2$, reflecting the hardware cost of the pipelined processing elements, systolic interconnections, and output registers. The power report shows a total dynamic power consumption of approximately 37.77 mW, with leakage power contributing a comparatively smaller portion. These results demonstrate that the proposed architecture achieves reasonable area and power efficiency for a systolic-array-based neural network accelerator.

Report : area	Report : power
Design : tiny_an_layer	Design : tiny_an_layer
Version : S-2021.06-SP5-1	Version : S-2021.06-SP5-1
Date : Thu Dec 4 06:51:10 2025	Date : Thu Dec 4 06:51:13 2025
Library(s) Used:	Library(s) Used:
NangateOpenCellLibrary (File: /synopsys/Nangate_FreePDK45/Ns	NangateOpenCellLibrary (File: /synopsys/Nangate_FreePDK45/NangateOpenCellLibrary_PDKv1_3_2010_12/Free
Number of ports: 2436	Number of ports: 2436
Number of nets: 38332	Number of nets: 38332
Number of cells: 30548	Number of cells: 30548
Number of combinational cells: 22580	Number of combinational cells: 22580
Number of sequential cells: 6976	Number of sequential cells: 6976
Number of macros/black boxes: 0	Number of macros/black boxes: 0
Number of buf/inv: 4593	Number of buf/inv: 4593
Number of references: 22	Number of references: 22
Combinational area: 42952.616413	Combinational area: 42952.616413
Buf/Inv area: 2607.332015	Buf/Inv area: 2607.332015
Noncombinational area: 34354.431671	Noncombinational area: 34354.431671
Macro/Black Box area: 0.000000	Macro/Black Box area: 0.000000
Net interconnect area: undefined (Wire load has zero net area)	Net interconnect area: undefined (Wire load has zero net area)
Total cell area: 77307.048084	Total cell area: 77307.048084
Total area: undefined	Total area: undefined

Fig. 7. Area(left) and power(right) reports obtained from Synopsys Design Compiler for the synthesized neural network accelerator.

Timing analysis plays a crucial role in determining the maximum achievable operating frequency. Before introducing pipelining, the worst-case critical path was dominated by the processing element MAC datapath, consisting of a long arithmetic chain involving logic gates and approximately 30 full adders in the accumulator. This resulted in limited timing slack and constrained the minimum clock period.

data arrival time	2.11		
clock clk (rise edge)	2.25	2.25	
clock network delay (ideal)	0.00	2.25	
clock uncertainty	-0.11	2.14	
ROW[0].COL[0].c_val_reg_reg[25]/CK (DFFR_X1)			0.00 2.14 r
library setup time	-0.02	2.11	
data required time	2.11		

data required time	2.11		
data arrival time	-2.11		

slack (MET)	0.00		

Fig. 8. Timing report after adding pipelining stage meeting slack and providing highest operating frequency of 444MHz

To improve performance, an additional pipeline stage was introduced inside the processing element to separate multiplication and accumulation. After pipelining, the critical path shifted away from the MAC and moved to the output stage, passing through the bias addition and ReLU logic. The post-pipelining timing report shows a data arrival time of approximately 2.11 ns with a worst-case slack of 0.00 ns at a clock period of 2.25 ns, meeting timing constraints. This confirms that pipelining significantly reduces the critical path length and improves the achievable operating frequency, making the design more efficient for high-throughput neural network inference.

C. Physical Design in Cadence Innovus

After successful synthesis, the generated gate-level netlist along with technology libraries and design constraints is imported into Cadence Innovus. The steps followed for performing Physical Design are shown in the flowchart below

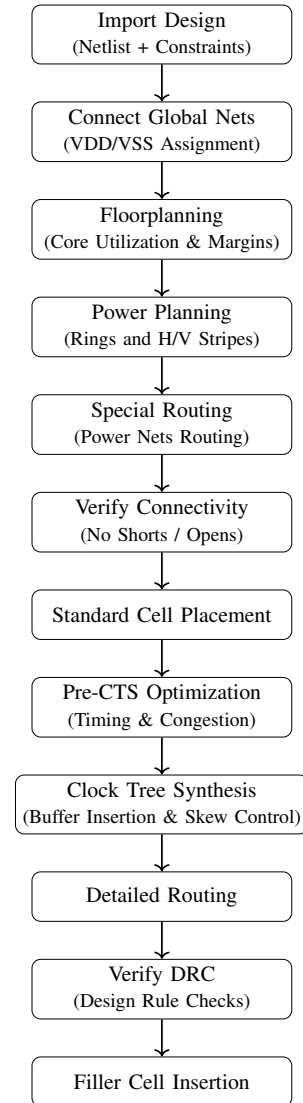


Fig. 9. Physical Design Flow in Cadence Innovus.

1) *Design Import and Global Net Connection:* The physical design flow begins by loading the synthesized netlist and constraints into Innovus. Global power and ground nets are then connected by assigning VDD to all power pins and VSS to all ground pins across the design. This ensures a consistent power reference for all standard cells.

2) *Floorplanning and Power Planning:* Floorplanning defines the physical structure of the chip. In this design, the floorplan is created with an aspect ratio of 1, a core utilization of 0.70, and uniform core margins of 10 μm on all sides. These parameters provide adequate routing resources and space for power grid insertion while maintaining area efficiency. Power planning is performed by adding power rings and power stripes to ensure robust power delivery. Power rings are created around the core using Metal 9 (horizontal) and Metal 10 (vertical) layers. Horizontal power stripes are inserted on Metal 7 with a width and spacing of 1 μm and a set-to-set distance of 25 μm , starting from the bottom of the core. Vertical power stripes are added on Metal 8 with identical width and spacing, starting from the left side of the core. This orthogonal stripe structure ensures uniform distribution of VDD and VSS across the design.

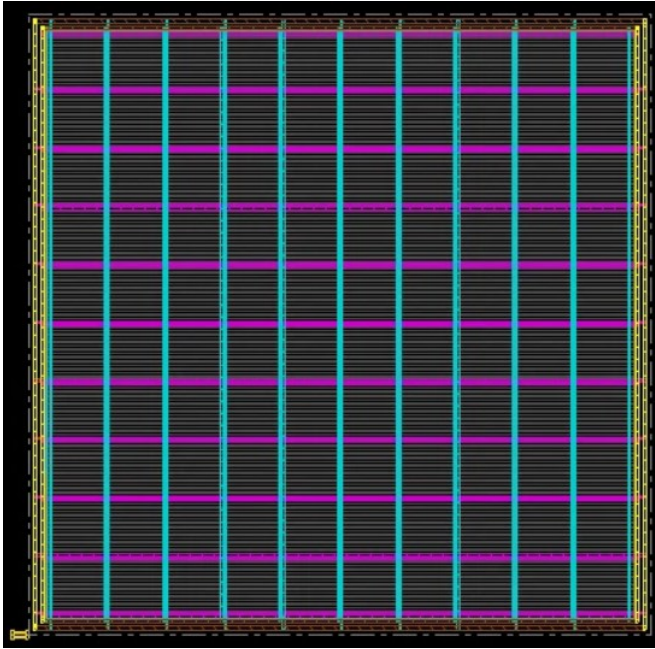


Fig. 10. Snippet after Floorplanning and Power Planning

3) *Special Routing and Connectivity Verification:* Special routing is applied to connect the power rings and stripes to the core and I/O structures. Connectivity checks are then performed to verify that there are no shorts or open connections in the power network. If violations are detected, special routing is repeated until clean connectivity is achieved.

4) *IO Pin Assignment and Connectivity Resolution:* The IO pins on the left side of the layout corresponding to the `a_col_bus[63:44]` signals were initially reported as unassigned nets. This occurred because the logical IO ports

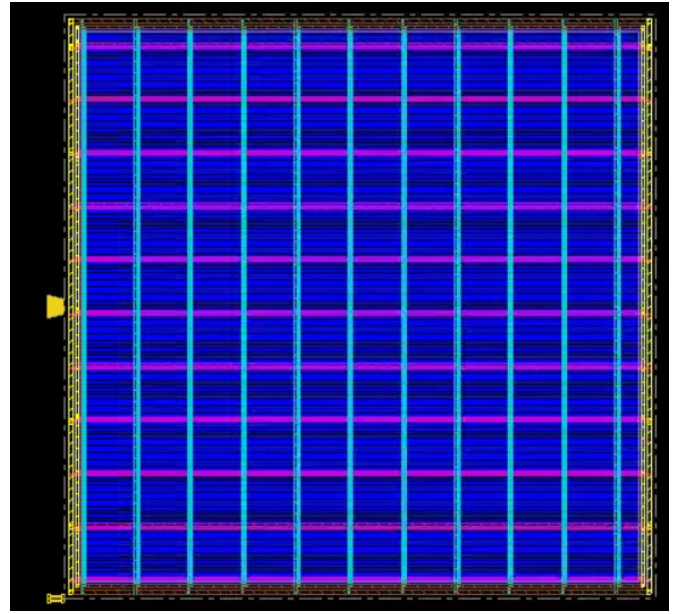


Fig. 11. Snippet after Special Routing

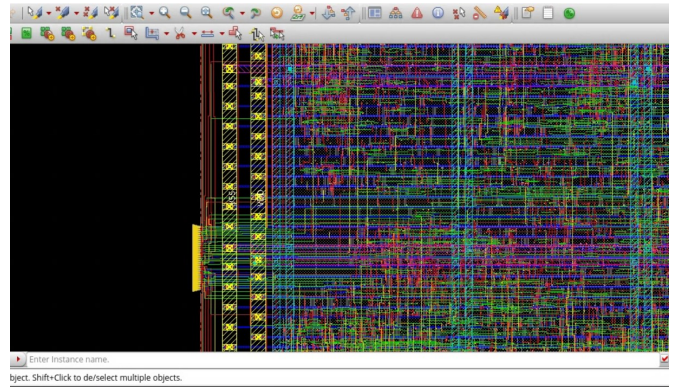


Fig. 12. IO Pads on left corner

defined in the RTL and netlist had not yet been physically placed on the die boundary. As a result, these nets did not have corresponding IO pin locations in the physical layout, and Cadence Innovus flagged them as unassigned during the `verifyConnectivity` check.

To resolve this issue, the IO pins for `a_col_bus[63:44]` were explicitly placed along the left edge of the die using the `editPin` command and were assigned to Metal 2 for routing. The remaining bus signals were placed along the bottom edge of the die to balance IO distribution. After assigning the IO pins and routing them on Metal 2, the connectivity checks were re-run and the unassigned-net warnings were eliminated, confirming that the logical IO ports were successfully mapped to physical IO locations on the chip boundary.

5) *Standard Cell Placement and Power Rail Analysis:* Standard cell placement is performed to position logic cells within the core while minimizing wirelength and congestion. Pre-CTS optimization is then carried out using ECO techniques.

A post-placement power report is generated using:

```
report_power > total_power.rpt
```

From Innovus, the total post-placement power is measured as $P_{\text{total}} = 24.05$ mW at a supply voltage of $V_{DD} = 0.95$ V. The supply current is calculated as:

$$I_{DD} = \frac{P_{\text{total}}}{V_{DD}} \quad (1)$$

$$I_{DD} \approx \frac{24.05 \times 10^{-3}}{0.95} \approx 25.3 \text{ mA} \quad (2)$$

The power grid consists of horizontal Metal 7 stripes and vertical Metal 8 stripes. The Metal 8 stripe width is $W = 1 \mu\text{m}$, and approximately $N = 11$ vertical stripes span the core height $L = 323.38 \mu\text{m}$. Using the Nangate 45 nm sheet resistance for Metal 8, $R_{\square} = 0.035 \Omega/\square$, the resistance of a single stripe is:

$$R_{\text{stripe}} = R_{\square} \left(\frac{L}{W} \right) \quad (3)$$

$$R_{\text{stripe}} \approx 11.29 \Omega \quad (4)$$

The equivalent resistance of parallel stripes is:

$$R_{\text{eq}} = \frac{R_{\text{stripe}}}{N} \approx 1.03 \Omega \quad (5)$$

The estimated IR drop is:

$$V_{\text{IR}} = I_{DD} \cdot R_{\text{eq}} \approx 26 \text{ mV} \quad (6)$$

The allowable 5% IR drop margin is:

$$V_{\text{IR,max}} = 0.05 \cdot V_{DD} = 47.5 \text{ mV} \quad (7)$$

Since $V_{\text{IR}} < V_{\text{IR,max}}$, the power grid is sufficient, and no further modification to stripe density is required.

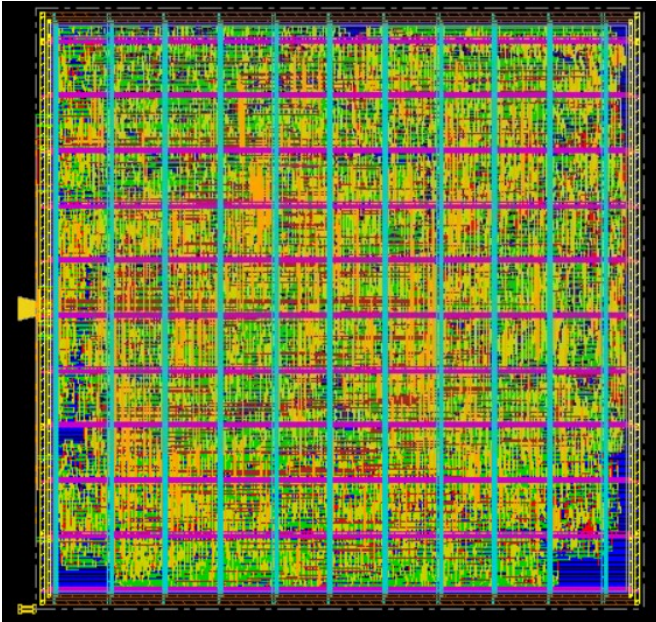


Fig. 13. Snippet after initial Placement

6) *Clock Tree Synthesis:* Clock Tree Synthesis (CTS) is executed using CCOpt to distribute the clock with controlled skew and latency. The following CTS commands are used:

```
set_ccopt_property cts_buffer_cells{BUF_X1 BUF_X2 INV_X1}
create_ccopt_clock_tree_spec
ccopt_design
```

These commands restrict the CTS engine to specific buffer and inverter cells and generate an optimized clock distribution network.

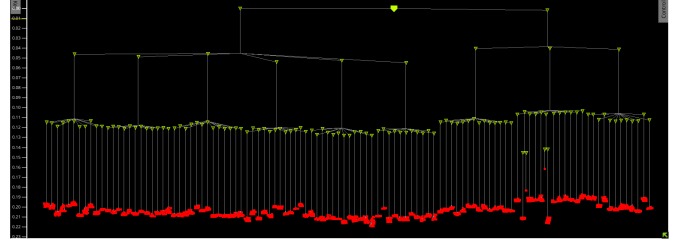


Fig. 14. Clock Tree of the implemented design

7) *Routing, Verification, and Filler Insertion:* After CTS, detailed routing is performed using NanoRoute. Post-route optimization is done, followed by design rule checks (DRC) to ensure layout correctness. Finally, filler cells (FILL1, FILL2, FILL4, FILL8, FILL16, and FILL32) are inserted to maintain well continuity and satisfy density requirements, completing the RTL-to-GDSII physical design flow.

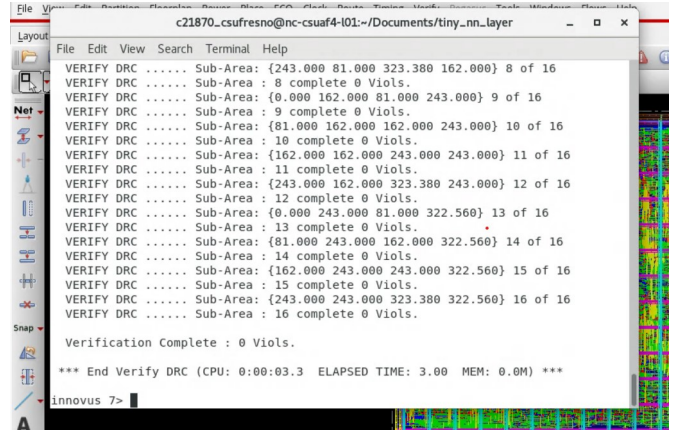


Fig. 15. Snippet showing DRC Check with no violations

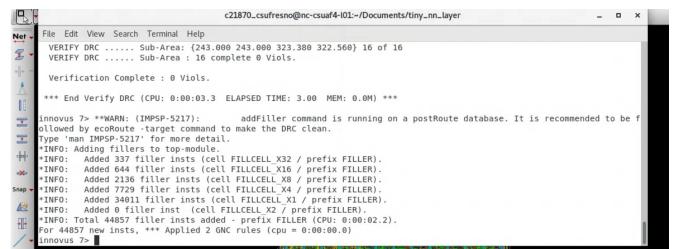


Fig. 16. Snippet after fillers are added

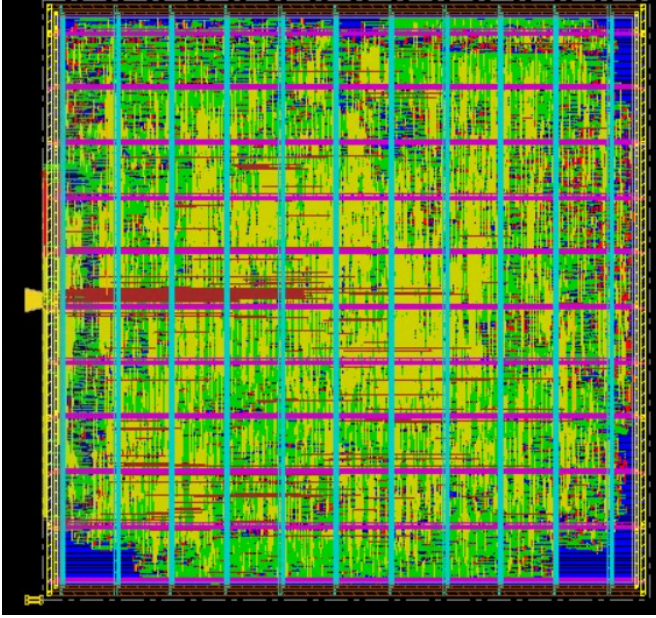


Fig. 17. Physical Design of Tiny Neural Network Accelerator

VI. CONCLUSION

This project successfully accomplished the complete RTL-to-GDSII implementation of a systolic-array-based neural network accelerator, demonstrating both functional correctness and physical feasibility. I attained efficient progress across RTL design, verification, synthesis, and physical design stages. The introduction and design sections established the necessary background on processing elements, systolic dataflow, and neural network operations, providing a strong foundation for the implementation.

Through systematic RTL development and functional verification, the design was validated to correctly perform matrix multiplication, bias addition, and ReLU activation. Synthesis and timing optimization, including the introduction of pipelining, significantly improved the critical path and enabled higher operating frequency. The physical design in Cadence Innovus further reinforced the practicality of the architecture, with careful floorplanning, power planning, clock tree synthesis, routing, and verification ensuring a clean and manufacturable layout.

Overall, this project highlights how architectural choices such as systolic arrays and pipelined MAC units translate into efficient silicon implementations. The experience gained through this work provides a solid understanding of the complete VLSI design flow and prepares me to tackle more complex hardware designs in future projects.

REFERENCES

- [1] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," *Sparse Matrix Proceedings*, pp. 256–282, 1979.
- [2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," *Proc. 44th Annual Int. Symp. Computer Architecture (ISCA)*, pp. 1–12, 2017.
- [3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [4] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," *Proc. Int. Conf. Machine Learning (ICML)*, pp. 807–814, 2010.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [6] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2010.
- [7] J. J. Zhang, K. Basu, and S. Garg, *Fault-Tolerant Systolic Array Based Accelerators for Deep Neural Network Execution*, *IEEE Design & Test*, vol. 36, no. 5, pp. 44–53, Oct. 2019.